

Module 12: LLM Foundations Master Interview Prep & Cheatsheet

This module consolidates all formulas, computational complexities, and model architecture trade-offs from the LLM Foundations curriculum into a single master study guide for Machine Learning and AI Engineer interviews.

1. Master Formula Sheet

1. TF-IDF Weights

$$\text{TF-IDF}(t, d, D) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \times \log \left(\frac{|D|}{1 + |\{d' \in D : t \in d'\}|} \right)$$

2. Word2Vec Negative Sampling (SGNS) Loss

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}_{w_O}^T \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}_{w_i}^T \mathbf{v}_{w_I})$$

3. Bahdanau (Additive) Attention

$$e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \quad \alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_k \exp(e_{i,k})}, \quad \mathbf{c}_i = \sum_j \alpha_{i,j} \mathbf{h}_j$$

4. Root Mean Square Normalization (RMSNorm)

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

5. SwiGLU Gated Activation

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x} \mathbf{W}_1) \odot \mathbf{x} \mathbf{W}_2) \mathbf{W}_3, \quad \text{where } \text{Swish}(a) = \frac{a}{1 + e^{-a}}$$

6. Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

7. Rotary Positional Embeddings (RoPE 2D Slice)

$$\mathbf{R}_{\Theta, m}^2 \mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

8. Attention with Linear Biases (ALiBi)

$$a_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i - j|\right)$$

9. KV Cache VRAM Size (FP16 Precision)

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

10. Chinchilla Scaling Compute-Optimal Training

$$D \approx 20 \cdot N, \quad C \approx 6ND \text{ FLOPS}$$

11. Perplexity (PPL)

$$\text{PPL}(X) = \exp\left(-\frac{1}{L} \sum_{t=1}^L \log P(x_t | x_{<t})\right)$$

2. Master Computational Complexity Sheet

Layer / Model Operation	Time Complexity	VRAM Space Complexity	Parallelizability
RNN Sequence Step	$O(L \cdot d^2)$	$O(L \cdot d)$	Sequential ($O(L)$ latency)
Self-Attention block	$O(L^2 \cdot d + L \cdot d^2)$	$O(L^2 \cdot h + L \cdot d)$	Fully parallel ($O(1)$ latency)
KV Cache Decoding Step	$O(L \cdot d + d^2)$	$O(b \cdot L \cdot d)$	Sequential (memory-bandwidth bound)
BPE Encoding Lookup	$O(L \cdot \log V)$	$O(V)$	Parallelizable
Top-p Nucleus Sort	$O(V \cdot \log V)$	$O(V)$	Non-parallelizable
FlashAttention Forward	$O(L^2 \cdot d)$	$O(L \cdot d)$ (HBM writes reduced)	Fully parallel (high MFU cache tiling)

3. Evolutionary Architectures Matrix

Architecture	GQA	RMSNorm	SwiGLU	Positional Encoding	Key Innovation
GPT-3	No	No (LayerNorm)	No (GELU)	Learned Absolute	Decoders scale to 175B
BERT	No	No (LayerNorm)	No (GELU)	Learned Absolute	Bidirectional MLM encoding
T5	No	No (LayerNorm)	No (ReLU)	Relative Bias	Unified Text-to-Text format
Llama-3	Yes	Yes	Yes	Rotary (RoPE)	Pre-normalization stability
Mistral	Yes	Yes	Yes	Rotary (RoPE)	Sliding Window Attention
DeepSeek	MLA	Yes	Yes (MoE)	Rotary (RoPE)	Latent Attention / Multi-token prediction

4. Interview Questions & Production Trade-offs

What problem does this solve?

Consolidates theoretical ML math intuition, complexity tables, and system bottlenecks into a single quick-reference sheet to enable rapid interview revision.

Why was it introduced?

Cracking high-level AI Engineer interviews requires connecting algorithmic choices (e.g. BPE vs WordPiece, GQA vs MHA) directly to production system constraints (VRAM, memory bandwidth, latency).

What are its limitations?

Cheatsheets summarize formulas and complexity classes, but cannot replace code-level debugging intuition (e.g., verifying PyTorch tensor sizes and gradients).

Computational Complexity (Time & Memory)

- **Time Complexity:** $O(1)$ lookup reference.
- **Memory Complexity:** $O(1)$ space.

Component Variable Denotation Legend

- N : Parameter count.
- L : Sequence token length.
- $|V|$: Vocabulary size.
- d : Hidden model dimension size.
- b : Batch size.
- n_{layers} : Transformer layer count.
- $n_{\text{heads_kv}}$: Key-Value heads count.
- d_{head} : Dimension per attention head (d/h).
- C : Total FLOPS compute budget.

Production Use Cases:

- Estimating cluster VRAM requirements during deployment scaling meetings.
- Structuring algorithmic justifications for architectural choices in engineering design reviews.

Follow-up Questions Interviewers Ask:

1. *Why does the backward pass of a parameter cost twice as much compute ($4N$) as the forward pass ($2N$) during LLM training?*
- **Answer:** The forward pass requires one matrix multiplication per layer to project inputs ($2N$ operations). The backward pass requires two separate matrix multiplications: one to calculate the gradients of the loss with respect to activations of the layer (for backpropagation down the stack), and a second to calculate the gradients of the loss with respect to the layer weights (to update parameters), resulting in $4N$ operations ($2 \times 2N$).
2. *Describe the exact mathematical relationship between Perplexity and Cross-Entropy Loss.*
- **Answer:** Perplexity is the exponentiated average cross-entropy loss: $\text{PPL} = e^{H(X)}$. If a model predicts tokens uniformly at random over a vocabulary of size $|V|$, its cross-entropy loss is $\log |V|$, and its perplexity is $e^{\log |V|} = |V|$. A lower loss scales perplexity down toward 1.0, indicating absolute certainty.
3. *Why does memory-bandwidth bound decode processing cause low GPU utilization (low SM occupancy) during single-user LLM inference?*
- **Answer:** Standard GPUs are optimized for massive parallel floating-point computation (high FLOPS). During single-user decoding, only a single token is generated per step. The GPU must load gigabytes of weights from HBM to SRAM for that single token. Since memory transfer speed is orders of magnitude slower than tensor core compute speed, the GPU execution units sit idle waiting for memory loads, leading to extremely low compute core utilization (5% – 15%).